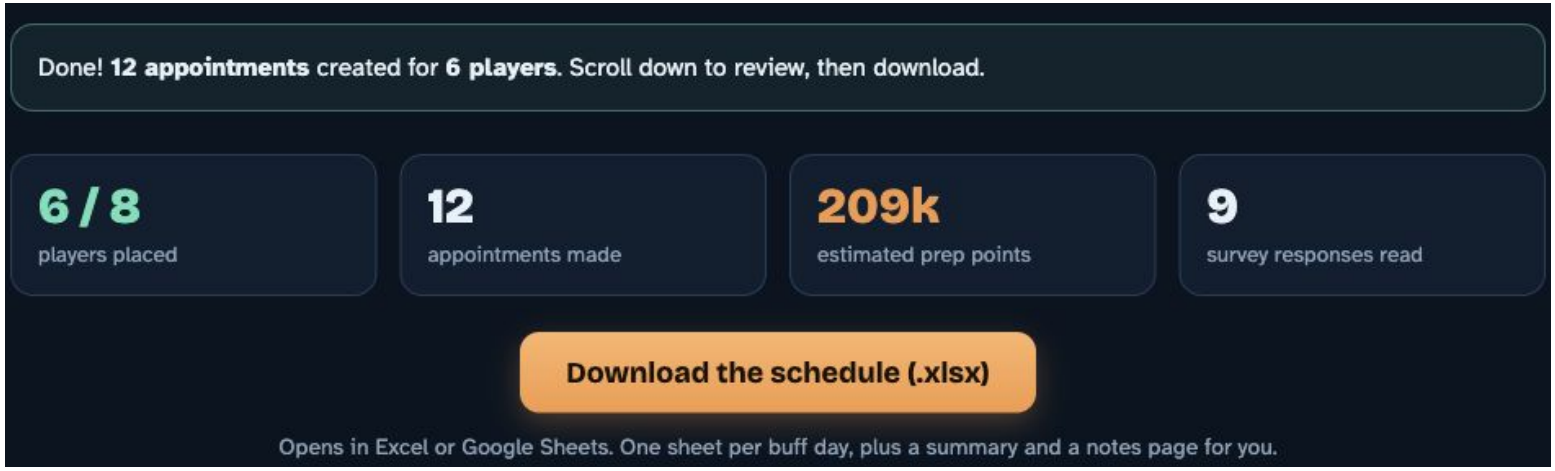
PROJECT CASE STUDY

A community schedule needs more than a ranked list.

The task combines player availability, shared resources and limited appointments across three event days.

Whiteout Survival communities arrange appointments for Construction, Research and Troop Training during an SvS preparation event. Players submit their available times, chosen days and resources through Microsoft Forms. An organizer then needs a schedule that uses those resources well without booking anyone outside their availability. A promising total is only useful if the resulting appointments can actually be kept.

I developed the scheduler, its browser interface, the Python workflow and the explanation of how its decisions work. The project connects data preparation, mathematical optimization and practical output for community organizers. I wanted someone to start with the survey they already collect, understand what the tool produced and see which answers still need attention. This case study follows those connections from an incoming response to a schedule someone can review.



A reviewed synthetic example: 9 responses become 8 unique player records and 12 appointments for 6 players.

Try the browser tool

PROJECT CASE STUDY

Reliable assignments begin with readable input.

The scheduler translates varied survey answers into consistent player records before it assigns a time.

A survey workbook is not a clean scheduling model. Column headings vary, resources arrive as numbers or written durations, and a player may submit more than once. The intake stage locates the relevant sheet and columns, recognizes Player IDs and builds one retained record per player. Completion timestamps guide duplicate handling when they are available, including timestamps stored as Excel serial values. Missing or unreadable availability does not become permission to schedule someone.

Resource parsing recognizes days, hours and minutes, then converts supported inputs into estimated points. Explicit units matter because a bare number defaults to days unless its header identifies another unit. Ambiguous amounts need clarification before use. Availability becomes a set of eligible appointment starts, with the full appointment required to fit inside the stated window. Selected days, resource values and comments remain distinct. This separation lets the organizer review an unusual answer without confusing a preference with a hard availability rule.



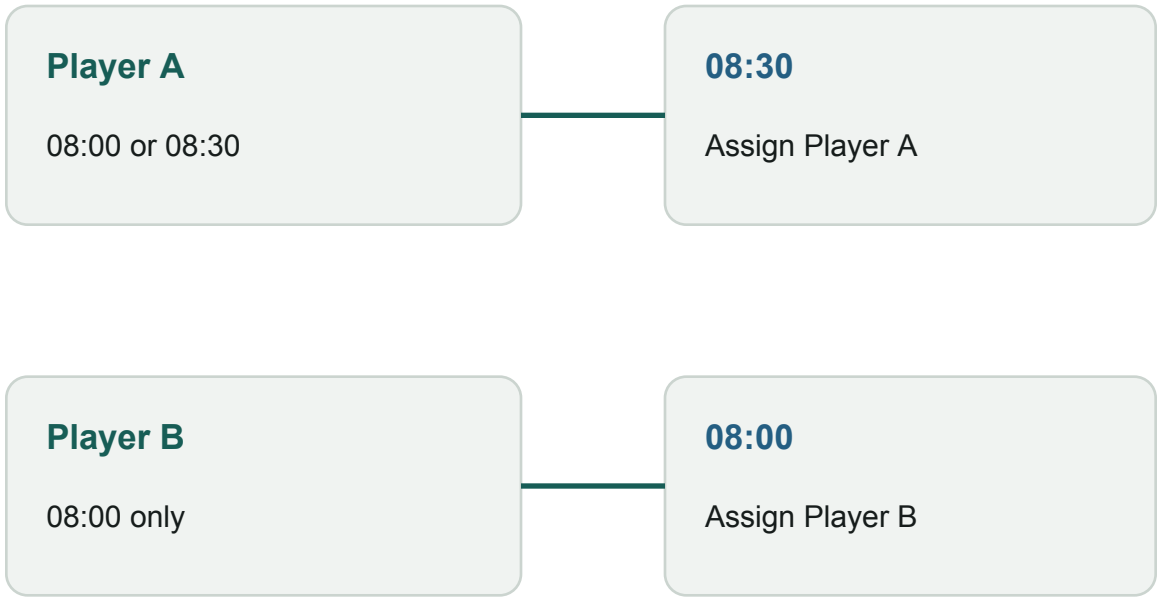
The record carries identity, selected days, resources, availability and comments as separate fields.

PROJECT CASE STUDY

A flexible player can protect a scarce appointment.

Choosing the largest contribution first can prevent a better arrangement for the whole event.

SYNTHETIC EXAMPLE / CONSTRUCTION DAY



The design decision

I model the connected problem across all three days. The solver can reconsider an earlier assignment when another arrangement improves the result. That ability to revise a choice is what the simple ranked approach lacks.

The rules

An appointment must fall on a selected day and within submitted availability. A slot holds at most one player. A player can receive at most one appointment on a day, with an additional limit across the event. These are constraints, not preferences the optimizer may trade away for more points.

A small example

Player A can attend at 08:00 or 08:30. Player B can attend only at 08:00. If A has the larger contribution, a simple ranking might give A the first slot and leave B out. Putting B at 08:00 and A at 08:30 includes both. The important information is how their options overlap, not just which player brings more.

PROJECT CASE STUDY

The objective order is built into the solver.

Estimated points come first, filled appointments second and preference rules only after those priorities.

The implementation represents appointments as flow through a network. A route passes through a player, a selected day and an available slot. Capacity limits control how many appointments can pass through each part. Missing connections rule out unavailable times. The minimum cost flow solver uses reverse connections to revise earlier choices as it finds a better arrangement. Integer capacities keep assignments whole, so the result does not depend on rounding a fraction of an appointment.

Integer weights preserve the priority order: maximize estimated points, then fill more slots, then apply the tie rules. The lower layer considers limited availability and signup order; the browser can also consider time requests in comments when that option is enabled. None of those preferences overrides availability or the first two objectives. Python integers and JavaScript BigInt retain the large weights exactly. A separate capacity limited connection credits each player's general speedup pool once across the event, rather than adding the same shared resources to every appointment.

1 / Estimated points

Use the recognized resources.

2 / Appointments filled

Prefer more appointments in a tie.

3 / Tie preferences

Consider wishes, scarcity and order.

Shared general speedups enter the score once per player across the event. Empty slots are allowed.

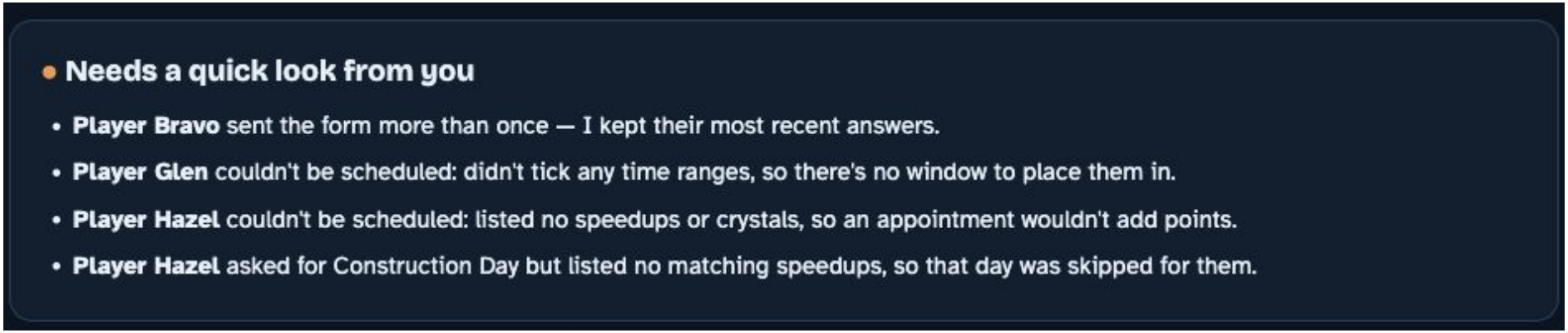
PROJECT CASE STUDY

Two routes make the result available for review.

The browser serves a straightforward file workflow, while Python supports inspection and existing schedule templates.

In the browser, an organizer loads the Forms export, reviews the results and notes, then downloads an Excel workbook. Before displaying results, the solver checks assignments against the hard rules. The download separates the three buff days, summarizes each player's appointments and includes Scheduler notes for unplaced players and comments. Spreadsheet data is processed locally, although the initial page load needs network access for its external libraries and fonts. Turning off player names does not anonymize Scheduler notes, so that sheet still needs review before sharing.

The Python route supports command line inspection and can fill an existing schedule template while preserving locked entries. Its output verifier reopens the written workbook and checks the assignments. The browser does not perform that same reopening step after download. Keeping the routes explicit makes the documentation more useful: organizers can choose a workflow, understand its checks and review the actual output without assuming both versions do everything identically.



The attention panel keeps issues visible. The workbook carries unplaced reasons and comments, but not every on page warning.

PROJECT CASE STUDY

Checks support the result and define its limits.

Observed behavior and recorded verification reports provide different kinds of evidence.

9	8	6	12
responses	unique players	players placed	appointments

Demonstrated

The synthetic walkthrough used nine responses representing eight unique players. The browser placed six players in twelve appointments and displayed 209k estimated points. Those figures describe this example. They do not measure community adoption, time saved, real event scores or improvement over a previous schedule.

Previously reported

The repository documentation reports comparisons with exhaustive enumeration on 400 Python cases and 150 JavaScript cases. Those are historical verification claims, not tests rerun for this publication. The reviewed public snapshot does not include the test files needed to reproduce those counts directly.

Still limited

The optimum applies to the parsed records, configured constraints and implemented point model. It cannot establish what a player will actually spend or whether an appointment will be kept. Unclear survey answers still need organizer judgment, and current game scoring rules need to agree with the constants used by the tool.

Source snapshot: 457e36e1fd5633fde36bb1ab7a529c73b91abaf8. Demonstration workbook reopened for this documentation review.

Inspect source and recorded verification

Read the algorithm explanation